

Relatório Técnico do Experimento RAG-InsurMinds

Aviso Importante

- Os arquivos oficiais deste projeto estão em: <https://github.com/marcelobazevedo/RAG-InsurMinds>
- Para executar a solução descrita neste relatório, é necessário baixar/clonar esse repositório.
- A execução da aplicação depende de ambiente Docker (Docker Engine + Docker Compose).

1. Resumo Executivo

Este relatório descreve o experimento de construção e consolidação de uma aplicação RAG (Retrieval-Augmented Generation) para o domínio de seguro residencial, implementada no repositório [RAG-InsurMinds](#).

A solução final entrega um fluxo operacional completo:

1. upload de documentos PDF;
2. ingestão automática e indexação vetorial em PostgreSQL + pgvector;
3. recuperação híbrida de contexto ([dense](#), [sparse](#), [hybrid](#));
4. geração de resposta em streaming com citação de fontes;
5. operação de limpeza completa (arquivos + base vetorial) via interface.

Para operação, os PDFs da pasta [documentos_para_upload/](#) devem ser enviados pela interface; após envio, ficam na pasta operacional [documentos/](#).

A arquitetura foi containerizada com Docker Compose, mantendo também execução local com [uv](#).

2. Objetivo do Experimento

O objetivo técnico foi validar um pipeline RAG de ponta a ponta que permitisse:

- transformar PDFs de negócio em base consultável;
 - responder perguntas com grounding em evidências recuperadas;
 - suportar operação simples por interface web;
 - manter flexibilidade de provedor de modelo (local via Ollama ou OpenAI);
 - garantir rastreabilidade da resposta por fonte.
-

2.1 Vídeo de Demonstração

Para registro da operação da aplicação, o experimento também possui um vídeo de demonstração:

- <https://drive.google.com/file/d/1GF7AtyAyOZ2fKrEI6nbRF6IB2XikERXc/view?usp=sharing>
-

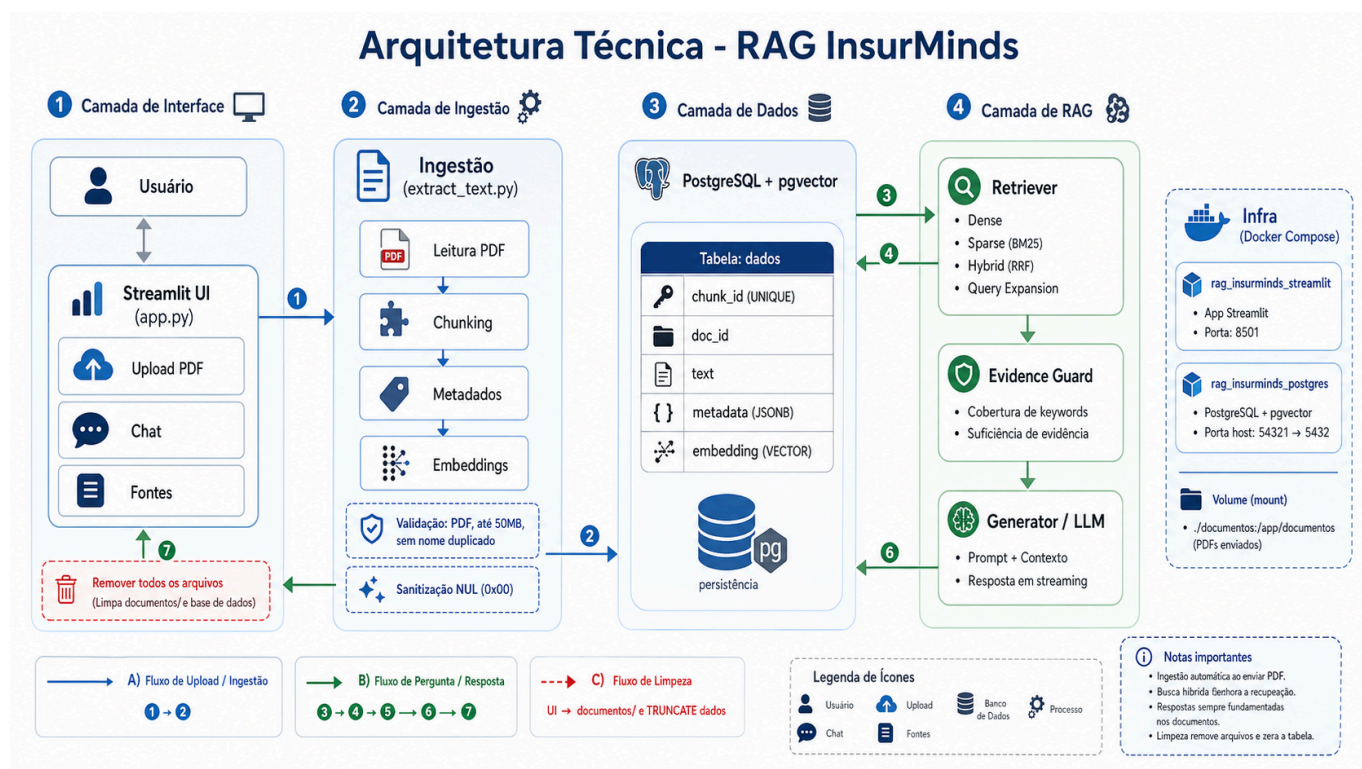
3. Escopo e Fontes de Verdade

A descrição deste relatório foi construída a partir dos artefatos do próprio projeto:

- [README.md](#)
- [TECNICO.md](#)
- [app.py](#)
- módulos em [rag/](#) (ingestão, recuperação, geração, guardrails)
- infraestrutura Docker ([Dockerfile](#), [docker-compose.yaml](#))
- diagrama técnico: [desenho_tecnico_rag_insurminds.png](#)

4. Arquitetura da Solução

A arquitetura está representada no diagrama [desenho_tecnico_rag_insurminds.png](#), com quatro camadas principais:



1. Camada de Interface

- Streamlit ([app.py](#)) com chat, upload e exibição de fontes.

2. Camada de Ingestão

- leitura de PDF;
- chunking com metadados;
- geração de embeddings;
- sanitização de texto/metadados;
- persistência no banco.

3. Camada de Dados

- PostgreSQL com extensão pgvector;
- tabela [dados](#) para texto + metadados + vetor.

4. Camada RAG

- recuperação **dense** (vetorial), **sparse** (BM25) e **hybrid** (RRF);
- guardrail de suficiência de evidência;
- geração de resposta por LLM com streaming.

4.1 Leitura do Desenho Técnico

O desenho técnico resume o funcionamento da solução como uma pipeline RAG de ponta a ponta. A leitura pode ser feita da esquerda para a direita, começando pela interação do usuário e avançando até a resposta fundamentada em fontes.

1. Entrada do usuário

- O usuário acessa a aplicação pela interface Streamlit.
- Pela mesma interface, ele pode enviar documentos PDF para compor a base de conhecimento ou fazer perguntas no chat.

2. Upload e armazenamento operacional dos PDFs

- Os arquivos enviados pela sidebar são validados pela aplicação.
- PDFs válidos são gravados na pasta **documentos/**, que funciona como área operacional para ingestão e também como origem dos links de fonte exibidos no chat.

3. Pipeline de ingestão

- Cada PDF salvo passa pelo processo de extração textual.
- O texto extraído é normalizado, dividido em seções quando marcadores do domínio são encontrados e depois segmentado em chunks menores.
- Cada chunk recebe metadados de rastreabilidade, como nome do PDF, **doc_id**, tipo de trecho, índice do chunk e informações inferidas do documento.

4. Geração de embeddings

- Para cada chunk textual, a aplicação gera um vetor de embedding.
- O provedor do embedding depende da configuração do ambiente: modelo local via Ollama quando **MODELO_LOCAL=true**, ou OpenAI quando **MODELO_LOCAL=false**.
- A dimensão do vetor deve ser compatível com **PGVECTOR_DIM**, pois essa dimensão define a coluna vetorial no banco.

5. Persistência no PostgreSQL + pgvector

- Os chunks, metadados e embeddings são gravados na tabela **dados**.
- A extensão pgvector permite armazenar e consultar vetores diretamente no PostgreSQL.
- O campo **chunk_id** atua como chave lógica para evitar duplicidade na ingestão.

6. Pergunta e recuperação de contexto

- Quando o usuário envia uma pergunta, o fluxo RAG consulta a base vetorial e textual.
- A recuperação pode ocorrer em modo **dense**, usando similaridade entre embeddings; em modo **sparse**, usando BM25; ou em modo **hybrid**, combinando os dois rankings por Reciprocal Rank Fusion.

- A etapa de query expansion pode reformular a pergunta com termos do domínio de seguro residencial para aumentar o recall.

7. Geração da resposta

- Os chunks recuperados são formatados como contexto e enviados ao LLM.
- O prompt orienta o modelo a responder apenas com base no contexto recuperado e a citar evidências no formato `[doc_id#chunk_id]`.
- A resposta é transmitida em streaming para melhorar a experiência de uso.

8. Exibição de fontes e rastreabilidade

- Ao final da geração, a interface exibe os documentos consultados.
- Os links apontam para os PDFs salvos em `documentos/`, permitindo verificar a origem das informações usadas na resposta.

9. Operações de manutenção

- O desenho também contempla a necessidade de manter consistência entre arquivos físicos e índice vetorial.
- A aplicação permite remover todos os PDFs e truncar a tabela `dados`; também há suporte no código para remover um documento específico e seus registros correspondentes.

As seções seguintes detalham tecnicamente cada bloco apresentado no desenho, desde a ingestão dos documentos até a geração da resposta com fontes.

4.2 Infraestrutura de execução

No `docker-compose` há dois serviços:

- `rag_insurminds_postgres` (pgvector);
- `rag_insurminds_streamlit` (aplicação).

Também foi configurado bind mount para documentos:

- `./documentos:/app/documentos`
- `./documentos:/app/static`

Isso garante que:

- arquivos enviados pela UI persistam no host;
- links de fontes possam abrir os PDFs via rota estática do Streamlit (`/app/static/...`).

5. Pipeline Implementada

5.1 Fluxo A - Upload e Ingestão

Fluxo efetivo implementado:

1. usuário envia PDF na sidebar;
2. validações de entrada:

- extensão `.pdf`;
 - tamanho máximo 50 MB;
 - bloqueio de nome duplicado;
3. arquivo (selecionado da pasta `documentos_para_upload/`) salvo em `documentos/`;
 4. ingestão automática do arquivo recém enviado;
 5. criação/garantia de tabela `dados`;
 6. chunking + metadados + embeddings;
 7. gravação no PostgreSQL com deduplicação por `chunk_id`.

5.1.1 Detalhes de ingestão

Em `extract_text.py`:

- extração textual com `pypdf`;
- divisão em chunks com `CHUNK_SIZE` e `CHUNK_OVERLAP`;
- tentativa de seccionamento por marcadores do domínio (`coberturas`, `exclusoes`, `franquia`, `sinistro`, `assistencia`);
- inferência de metadados como `doc_id`, `titulo`, `numero_documento`, `pdf_name`, `chunk_type` e `chunk_index`.

Em `pgvector_store.py`:

- tabela `dados` com coluna `embedding VECTOR(PGVECTOR_DIM)`;
- inserção com `ON CONFLICT (chunk_id) DO NOTHING`;
- sanitização de `\x00` (NUL) para evitar falhas de escrita no banco.

5.2 Fluxo B - Pergunta e Resposta

Fluxo efetivo implementado:

1. usuário envia pergunta;
2. `run_streaming_rag()` inicia execução do grafo;
3. nó `retrieve` retorna documentos recuperados;
4. nó `generate` gera resposta em streaming;
5. UI exibe tokens em tempo real e lista de fontes.

5.2.1 Recuperação

Em `retriever.py`:

- `dense_search`: similaridade vetorial por produto interno;
- `bm25_search`: ranking lexical com BM25;
- `hybrid_search`: fusão via Reciprocal Rank Fusion.

Em `retrieval_node.py`:

- expansão opcional de query com LLM, especializada em termos de seguro residencial;
- recuperação para modos `dense`, `sparse`, `hybrid`;
- retorno de metadados de busca para painel de detalhes da UI.

5.2.2 Geração e Grounding

Em `augmented_node.py`:

- prompt com política estrita de grounding;
- instrução explícita para citação em formato `[doc_id#chunk_id]`;
- fallback textual quando não houver base.

5.3 Fluxo C - Limpeza Operacional

A UI inclui botão “Remover todos os arquivos”, que executa:

1. remoção de PDFs em `documentos/`;
2. `TRUNCATE TABLE dados RESTART IDENTITY` no PostgreSQL.

Esse fluxo reduz inconsistência entre filesystem e índice vetorial.

6. Modelo de Dados

Tabela principal: `dados`

Campos:

- `id` (serial, PK)
- `chunk_id` (text, unique)
- `doc_id` (text)
- `titulo` (text)
- `fonte` (text)
- `data` (text)
- `tipo` (text)
- `text` (text)
- `metadata` (jsonb)
- `embedding` (vector (PGVECTOR_DIM))

Observações:

- `chunk_id` funciona como chave lógica para deduplicação;
 - `metadata` concentra rastreabilidade de origem;
 - a dimensão do vetor deve ser compatível com `PGVECTOR_DIM`.
-

7. Estratégia de Modelos (Local e OpenAI)

Em `model_provider.py`, a escolha do backend é controlada por `MODELO_LOCAL`.

7.1 Modo local (`MODELO_LOCAL=true`)

- chat e embedding via Ollama;
- variáveis: `LLM_MODEL`, `EMBEDDING_MODEL`, `OLLAMA_URL`.

7.2 Modo OpenAI (`MODELO_LOCAL=false`)

- chat via `OPENAI_MODEL`;
- embedding via `OPENAI_EMBEDDING_MODEL`;
- autenticação por `OPENAI_API_KEY` em `.env`.

Validações de variáveis obrigatórias são feitas antes da inferência.

8. Reprodutibilidade e Operação

8.1 Execução em container

Comandos principais:

```
docker compose up -d --build
docker compose ps
docker compose logs -f streamlit
docker compose logs -f postgres
```

8.2 Execução local com uv

```
uv sync
uv run streamlit run app.py
```

8.3 Ingestão manual (opcional)

```
docker compose exec streamlit uv run --no-sync python -m
rag.ingest.extract_text
```

9. Decisões Técnicas Relevantes

1. Escolha por recuperação híbrida (`dense` + `sparse` + RRF)
 - melhora robustez em perguntas semânticas e literais.
2. Ingestão automática no upload
 - reduz atrito operacional para usuários não técnicos.
3. Sanitização de caracteres NUL
 - corrige falha comum de PDFs malformados.
4. Limpeza unificada (arquivos + banco)

- mantém consistência entre conteúdo físico e índice vetorial.

5. Exibição de fontes por resposta

- melhora auditabilidade e confiança no resultado.
-

10. Limitações Observadas

1. Custo/latência de embeddings em tempo de upload

- em documentos grandes, ingestão pode demorar proporcionalmente ao número de chunks.

2. Chunking heurístico

- baseado em marcadores textuais; PDFs muito desestruturados podem reduzir qualidade da segmentação.

3. Guardrail desacoplado da resposta final no frontend

- o módulo de avaliação de suficiência existe no código, mas o ciclo de UX atual privilegia resposta + fontes sem painel específico de decisão do guardrail.

4. Dependência da qualidade do OCR/texto original

- documentos escaneados ou com extração ruim podem prejudicar recall e resposta.
-

11. Riscos e Mitigações

1. Erro de dimensão de embedding

- mitigado por validação explícita em `PgVectorStore.add_texts()`.

2. Erro de conexão com banco

- mitigado por documentação operacional e separação de portas host/container.

3. Link de fonte quebrado

- mitigado por mount de `documentos` também em `/app/static` e construção de link compatível com Streamlit.

4. Respostas sem evidência forte

- mitigado por política de grounding, citação obrigatória e heurísticas de suficiência.
-

12. Conclusão

O experimento atingiu o objetivo de disponibilizar um RAG funcional, operacional e rastreável para o domínio de seguro residencial, com:

- pipeline automatizada de ingestão;

- indexação vetorial em pgvector;
- recuperação híbrida;
- geração com grounding e citação;
- operação simplificada por interface e containerização.

A solução está pronta para uso interno e para evolução incremental (ex.: frontend React/API dedicada, observabilidade, métricas online e avaliação contínua com dataset curado do domínio).

13. Artefatos Principais

- Aplicação: [app.py](#)
- Ingestão: [extract_text.py](#), [pgvector_store.py](#)
- Recuperação: [retriever.py](#), [retrieval_node.py](#)
- Geração: [augmented_node.py](#)
- Model provider: [model_provider.py](#)
- Infra: [docker-compose.yaml](#), [Dockerfile](#)
- Documentação base: [README.md](#), [TECNICO.md](#)
- Figura de arquitetura: [desenho_tecnico_rag_insurminds.png](#)